PROJECT REPORT

LifeVault

Personal Safety and Diary Vault using C Program

Project Theme

A console-based C application that combines emergency information, personal diary records, future messages, memory preservation, legacy notes, and goal tracking into one private LifeVault concept.

Submitted By	V. SHAKTHI K. SHARMITHA A.C SHRI SUNDARAM
Submitted To	Dr. T. BALAKUMARAN
Department	Department of Electronics and Communication Engineering
Institution	Coimbatore Institute of Technology
Date of Submission	04-05-2026

GitHub Repository: https://github.com/shrisundaram2120/lifevault

Live Prototype: https://shrisundaram2120.github.io/lifevault/

Abstract

LifeVault is a personal safety and life-management vault developed using the C programming language. The project is designed as a private console application where users can create a vault, log in using Gmail with their preferred passcode or password, maintain an emergency profile, and store meaningful personal records. The main record section works as a diary-based vault, where the user can add different types of records such as life events, future messages, dream signals, legacy instructions, memories, and goals.

The application uses structures to organize user details, emergency information, and diary records. It uses pointers to pass structures efficiently between functions and file pointers for persistent storage. User login details are saved in vault_passcode.txt, remembered login information is saved in remembered_user.txt, and diary records are saved in lifevault_records.txt. The current C version uses the DD-MM-YYYY date format and supports future-locked records that are hidden until the selected unlock date.

A prototype website is also provided to visually demonstrate the expanded LifeVault idea. The website includes separate portals for diary pages, future messages, legacy instructions, memory cards, goals, and insights. The C application remains the main assignment program, while the website helps explain the interface and future development scope.

Table of Contents

1. Introduction

In daily life, people store important personal information in different places. Emergency contact details, personal thoughts, future messages, memories, family instructions, and goals are often scattered across notebooks, mobile notes, chats, or separate applications. LifeVault brings these ideas into one meaningful C project. The application is not only a storage system; it is presented as a personal vault that protects important life information and helps the user revisit meaningful records.

The project is useful for demonstrating core C programming concepts such as structures, arrays, strings, functions, pointers, file handling, validation, and menu-driven programming. It is also more unique than common C assignment topics such as bank management, library management, inventory systems, or simple voting systems because it connects technical implementation with a human-centered purpose.

The current implementation focuses on a command-line interface. A user can view the emergency card without entering the vault, create a new vault account, log in as a remembered user, or log in using Gmail. After unlocking the vault, the user can update emergency details, add diary records, view the diary timeline, analyze patterns, and display a vault report.

2. Problem Statement

Many personal management systems focus only on one task, such as notes, reminders, contacts, or goals. In emergency situations, important personal details may not be quickly available. At the same time, meaningful personal records such as memories, future messages, legacy instructions, and goals may not be stored in a structured way. The problem is to create a simple but meaningful C application that can store and manage these different life records in one private vault.

The system should support user login, emergency information, diary-style record creation, future date locking, timeline viewing, pattern analysis, and permanent file storage. The design should also be easy to explain, practical for a C programming assignment, and unique enough to stand out from routine management-system projects.

3. Objectives

- To build a menu-driven personal vault application using C.
- To allow different users to create a vault using Gmail and a passcode or password preference.
- To store login information in vault_passcode.txt and remember the last user through remembered_user.txt.
- To maintain emergency profile information that can be viewed quickly without full vault navigation.
- To create diary records with a title, date, type, mood, tags, type-specific questions, and an extra diary note.
- To support future messages that remain hidden until the selected DD-MM-YYYY unlock date.
- To analyze diary patterns using mood words, tags, future-lock data, and reflective keywords.
- To demonstrate structures, pointers, arrays, strings, functions, and file handling clearly.

4. Existing System and Proposed System

Aspect	Existing/Common Systems	Proposed LifeVault System
Purpose	Usually handles one domain such as bank accounts, library books, voting, or inventory.	Combines emergency safety, diary records, future messages, memories, legacy notes, and goals.
User Value	Mostly administrative or transaction based.	Personal, emotional, practical, and safety-focused.

Aspect	Existing/Common Systems	Proposed LifeVault System
Record Style	Records are usually fixed-format forms.	Diary records change their questions based on the selected record type.
Privacy	Many beginner systems have no user identity or only one fixed password.	Users join with Gmail and select their own passcode or password.
Unique Feature	Simple add, view, delete or search operations.	Future entries can be locked until a selected date, and the analyzer marks entries worth revisiting.

5. System Modules

Module	Description
Emergency Profile Module	Stores and displays important emergency details such as full name, blood group, allergies, medical notes, emergency contact, contact phone number, and routine. In the current C version, this information is available during program execution.
Join / Create Vault Module	Allows a new user to enter name, Gmail, secret type, and a passcode or password. The data is appended to vault_passcode.txt.
Remembered User Login Module	Stores the most recent Gmail in remembered_user.txt. The next time the program runs, the user can log in quickly by entering only the correct secret.
Gmail Login Module	Searches vault_passcode.txt using the entered Gmail and verifies the matching passcode or password.
Diary Life Record Module	Lets the user create private diary records with title, type, mood, tags, date, answers, and an extra diary note.
Future Lock Module	For future messages, the user enters an unlock date. The record stays hidden in the timeline until the date is reached.
Diary Timeline Module	Displays saved records in memory during the current session and hides records that are still future locked.
Diary Pattern Analyzer	Counts total entries, tagged entries, locked future entries, and reflective entries that may need revisiting.
Vault Report Module	Displays summary statistics such as total entries, future messages, and entries worth revisiting.
File Handling Module	Uses text files for login and diary persistence. The main record file is lifevault_records.txt.

6. Algorithm and Working Flow

Main Menu Algorithm

1. Start the program and initialize the emergency profile.
2. Display the LifeVault main menu with options for emergency card, creating a vault, remembered-user login, Gmail login, and exit.
3. If the user chooses emergency card, display the emergency profile directly.
4. If the user creates a vault, collect name, Gmail, secret type, and secret value, then save it to vault_passcode.txt.
5. If the user logs in successfully, open the vault menu.
6. Continue until the user chooses exit.

Vault Menu Algorithm

1. Show the unlocked vault menu after successful login.
2. Allow the user to update or display the emergency profile.
3. Allow the user to add a new diary life record.
4. Allow the user to view the diary timeline. Future locked entries are checked before display.
5. Allow the user to analyze diary patterns and display a vault report.
6. Exit the vault menu when the user chooses to lock the vault.

Add Diary Record Algorithm

1. Check whether the maximum record count has been reached.
2. Read title, entry date, record type, mood, tags, answers to type-specific questions, and extra diary note.
3. If the type is Future Message, read the unlock date in DD-MM-YYYY format.
4. Store the record in the LifeRecord structure array.
5. Append the record to lifevault_records.txt using file handling.

Future Lock Algorithm

1. Convert the unlock date from DD-MM-YYYY to a comparable numeric value.
2. Get the current date and convert it to the same numeric format.
3. If the current date is before the unlock date, hide the record from the timeline.
4. If the current date is equal to or after the unlock date, allow the record to be displayed.

7. Data Structures, Pointers and File Handling

LifeVault uses structures to group related data. This makes the program clearer than using many separate variables. It also demonstrates pointer usage because structures are passed to functions using pointer parameters such as EmergencyProfile *profile, VaultUser *user, and LifeRecord *record. C does not have constructors like C++, so initialization is handled through functions such as initializeEmergencyProfile().

Main Structures

```
typedef struct {
    char fullName[NAME_SIZE];
    char bloodGroup[NAME_SIZE];
    char allergies[TEXT_SIZE];
    char medicalNotes[TEXT_SIZE];
    char contactName[NAME_SIZE];
    char contactPhone[NAME_SIZE];
    char routine[TEXT_SIZE];
} EmergencyProfile;

typedef struct {
    char title[TITLE_SIZE];
    char type[NAME_SIZE];
    char mood[NAME_SIZE];
    char tags[TEXT_SIZE];
    char entryDate[DATE_SIZE];
    char unlockDate[DATE_SIZE];
    char questionOne[QUESTION_SIZE];
    char answerOne[TEXT_SIZE];
    char questionTwo[QUESTION_SIZE];
    char answerTwo[TEXT_SIZE];
    char questionThree[QUESTION_SIZE];
    char answerThree[TEXT_SIZE];
    char diaryBody[TEXT_SIZE];
} LifeRecord;

typedef struct {
    char name[NAME_SIZE];
    char gmail[GMAIL_SIZE];
    char secretType[NAME_SIZE];
    char secret[SECRET_SIZE];
} VaultUser;
```

Important File Names

File Name	Purpose in Current C Program
vault_passcode.txt	Stores user login data in the format name|gmail|secretType|secret.
remembered_user.txt	Stores the Gmail of the remembered user for quicker login.
lifevault_records.txt	Stores diary life records permanently by appending each saved record.
lifevault.c	Main C source code containing structures, functions, menus, analyzer, and file handling.

File Handling Used

The program uses file pointers such as FILE *file and functions such as fopen(), fprintf(), fgets(), fclose(), and strtok(). Login data is appended or searched from vault_passcode.txt. Diary records are appended to lifevault_records.txt. This satisfies the file-handling requirement of the C assignment while keeping the format simple enough to inspect manually.

```
#define FILE_NAME "lifevault_records.txt"
#define PASSCODE_FILE "vault_passcode.txt"
#define REMEMBERED_FILE "remembered_user.txt"

FILE *file = fopen(FILE_NAME, "a");
fprintf(file, "[Diary Life Record]
");
```

```
    fclose(file);
```

8. Diary Pattern Analyzer Logic

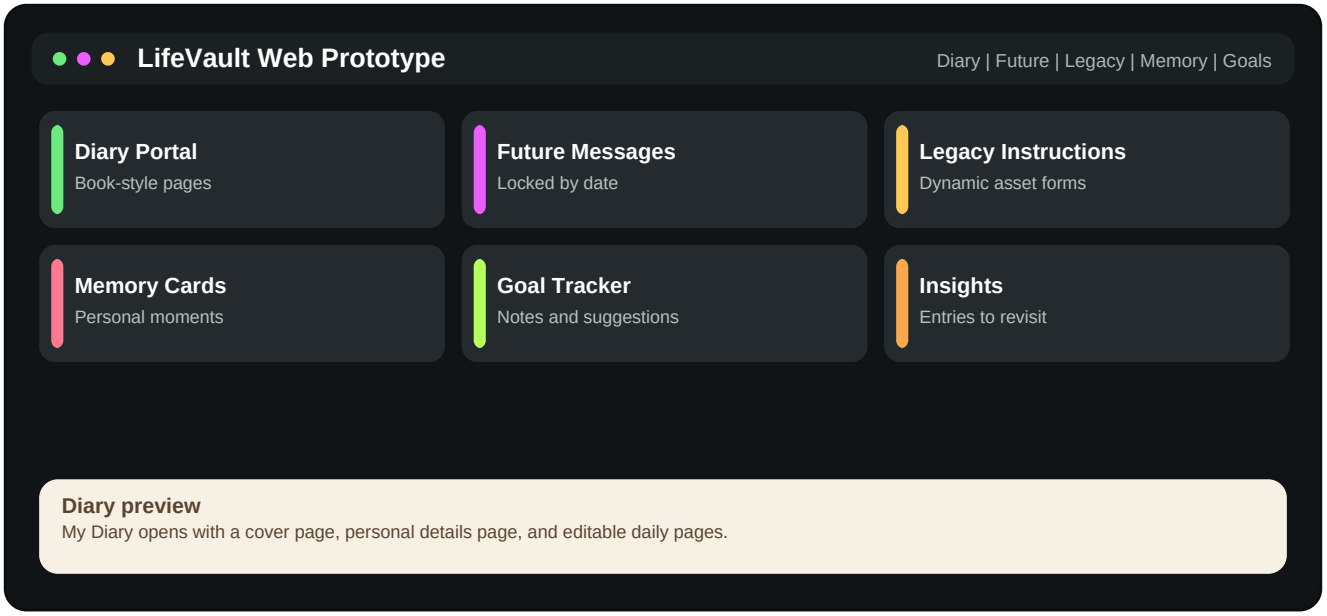
The current LifeVault program does not use numeric stress level, energy level, outcome, or a mathematical risk-score formula. Instead, it uses a more natural diary-based analysis method. Since the user enters mood as text, such as Calm, Stressed, Low, Happy, or Confused, the analyzer checks meaningful words in the mood and diary answers.

Analyzer Count	How It Is Calculated
Total diary entries	Counts all diary records stored in the current session.
Entries with tags	Counts records where the tags field is not empty.
Locked future entries	Counts future messages that still have a future unlock date.
Entries to revisit	Checks whether mood or diary text contains reflective words such as anxious, stressed, angry, low, confused, warning, health, regret, mistake, urgent, or hurt.

This approach is suitable for the project because it matches diary writing better than asking the user to enter numbers for stress or energy. It also makes the system feel more personal and realistic.

9. Prototype Website and Output Evidence

Along with the C assignment version, a live website prototype is available to show how LifeVault can look as a modern application. The website stores data in the browser using localStorage, while the C program stores data in text files. The prototype is useful for presentation and demonstration, but the C program remains the main assignment implementation.



Prototype Section	Purpose
Diary Portal	Book-like diary interface with front page, cover page, personal details page, and unlimited editable diary pages.
Future Message Portal	Stores messages that remain hidden until the selected date and time.
Legacy Instruction Portal	Supports multiple assets such as land, house, property, vehicle, jewellery, bank note, and other belongings with dynamic questions.
Memory Portal	Stores important memories in memory-card style records.
Goal Portal	Tracks goals, progress notes, initiatives, and suggested next steps.

Repository Link: https://github.com/shrisundaram2120/lifevault

Live Website Link: https://shrisundaram2120.github.io/lifevault/

Sample Console Output

```
LIFEVault MAIN MENU
1. View Emergency Card
2. Join / Create Vault
3. Login Remembered User
4. Login with Gmail
5. Exit

VAULT MENU
1. Update Emergency Profile   2. Show Emergency Card
3. Add Diary Life Record      4. View Diary Timeline
5. Analyze Diary Patterns     6. Display Vault Report
```

10. Advantages, Limitations and Future Scope

Advantages

- Unique project concept compared with common bank, library, voting, or inventory systems.
- Uses structures, pointers, arrays, strings, functions, and file handling in a meaningful way.
- Supports multiple users through Gmail-based login records.
- Allows the user to select passcode or password as the preferred secret type.
- Stores diary records permanently in a text file.
- Future messages can be hidden until a selected unlock date.
- Diary questions change based on record type, making the system more personal and less repetitive.
- Website prototype improves presentation value and helps others understand the final vision.

Limitations

- The C version is a console application, so it does not provide a graphical interface.
- Login secrets are stored in plain text for assignment simplicity; real systems should use hashing or encryption.
- Emergency profile information is maintained during the program session in the current C version and can be made file-persistent in a later version.
- The C program stores text-based records and does not directly store photo, audio, or video attachments.
- The analyzer uses keyword-based logic, so it cannot fully understand human emotions like an advanced AI system.

Future Scope

- Add encrypted storage for vault secrets and private records.
- Save the emergency profile permanently in a dedicated emergency_card.txt file.
- Add editing and deleting of saved records directly from the C application.
- Add search and filter options for records by date, type, mood, and tags.
- Add multimedia attachment support in the full application version.
- Add reminder notifications for goals and future messages.
- Connect the C logic with a database-backed web or mobile interface.

11. Conclusion

LifeVault is a unique and useful C programming project that combines personal safety, diary writing, future messages, memory preservation, legacy instructions, and goal tracking into one concept. The current C implementation demonstrates important programming concepts such as structures, pointers, arrays, string handling, menu-driven logic, date comparison, and file handling.

The project stands out because it is not only a data-management system but also a personal life-management vault. It can be explained as a practical application for storing meaningful life information safely and revisiting important records when needed. With future improvements such as encryption, persistent emergency-card storage, editing features, and database support, LifeVault can be expanded into a stronger real-world personal vault application.

Final Consistency Note

This updated report matches the current LifeVault C program: records are stored in lifevault_records.txt, login details are stored in vault_passcode.txt, remembered login uses remembered_user.txt, dates use DD-MM-YYYY, mood is text-based, and the analyzer uses keyword-based diary pattern detection instead of numeric stress or risk-score formulas.

Appendix A: Complete Source Code

File included: lifevault.c

The full C source code is placed after the main report so the explanation remains readable.

Line	Code

```c
#include <ctype.h>
#include <stdio.h>
#include <string.h>
#include <time.h>

#define MAX_RECORDS 80
#define TEXT_SIZE 300
#define TITLE_SIZE 80
#define NAME_SIZE 60
#define GMAIL_SIZE 80
#define SECRET_SIZE 40
#define DATE_SIZE 12
#define QUESTION_SIZE 90
#define FILE_NAME "lifevault_records.txt"
#define PASSCODE_FILE "vault_passcode.txt"
#define REMEMBERED_FILE "remembered_user.txt"

typedef struct {
    char fullName[NAME_SIZE];
    char bloodGroup[20];
    char allergies[TEXT_SIZE];
    char medicalNotes[TEXT_SIZE];
    char contactName[NAME_SIZE];
    char contactPhone[30];
    char routine[TEXT_SIZE];
} EmergencyProfile;

typedef struct {
    char title[TITLE_SIZE];
    char type[40];
    char mood[30];
    char tags[TEXT_SIZE];
    char entryDate[DATE_SIZE];
    char unlockDate[DATE_SIZE];
    char questionOne[QUESTION_SIZE];
    char answerOne[TEXT_SIZE];
    char questionTwo[QUESTION_SIZE];
    char answerTwo[TEXT_SIZE];
    char questionThree[QUESTION_SIZE];
    char answerThree[TEXT_SIZE];
    char diaryBody[TEXT_SIZE];
} LifeRecord;

typedef struct {
    char name[NAME_SIZE];
    char gmail[GMAIL_SIZE];
    char secretType[20];
    char secret[SECRET_SIZE];
} VaultUser;

void clearInputBuffer(void);
void readLine(char text[], int size);
void initializeEmergencyProfile(EmergencyProfile *profile);
int joinVault(VaultUser *user);
int loginRememberedUser(VaultUser *user);
int loginWithGmail(VaultUser *user);
int findUserByGmail(const char gmail[], VaultUser *user);
void saveRememberedUser(const char gmail[]);
int loadRememberedUser(VaultUser *user);
void createEmergencyProfile(EmergencyProfile *profile);
void showEmergencyProfile(const EmergencyProfile *profile);
void addLifeRecord(LifeRecord records[], int *recordCount);
void chooseRecordType(LifeRecord *record);
void setTypeQuestions(LifeRecord *record);
void viewLifeTimeline(LifeRecord records[], int recordCount);
void analyzeDiaryPatterns(const LifeRecord records[], int recordCount);
void displayVaultReport(const LifeRecord records[], int recordCount);
void saveRecordToFile(const LifeRecord *record);
int isReflectiveRecord(const LifeRecord *record);
int containsIgnoreCase(const char text[], const char keyword[]);
int isRecordLocked(const LifeRecord *record);
int dateToNumber(const char date[]);
int todayToNumber(void);
void todayToDmy(char date[]);

int main(void) {
    EmergencyProfile profile;
    VaultUser activeUser;
    LifeRecord records[MAX_RECORDS];
    int recordCount = 0;
    int startChoice = 0;
    int choice = 0;
    int unlocked = 0;

    initializeEmergencyProfile(&profile);

    do {
```

Line	Code

```
88          printf("====================================\n");
89          printf("              LIFEVAULT\n");
90          printf(" Personal Safety and Diary Vault\n");
91          printf("====================================\n");
92          printf("1. View Emergency Card\n");
93          printf("2. Join / Create Vault\n");
94          printf("3. Login Remembered User\n");
95          printf("4. Login with Gmail\n");
96          printf("5. Exit\n");
97          printf("Enter your choice: ");
98
99          if (scanf("%d", &startChoice) != 1) {
100             printf("Invalid input.\n");
101             clearInputBuffer();
102             continue;
103         }
104         clearInputBuffer();
105
106         if (startChoice == 1) {
107             showEmergencyProfile(&profile);
108         } else if (startChoice == 2) {
109             if (joinVault(&activeUser)) {
110                 unlocked = 1;
111             }
112         } else if (startChoice == 3) {
113             if (loginRememberedUser(&activeUser)) {
114                 unlocked = 1;
115             }
116         } else if (startChoice == 4) {
117             if (loginWithGmail(&activeUser)) {
118                 unlocked = 1;
119             }
120         } else if (startChoice == 5) {
121             printf("LifeVault closed.\n");
122             return 0;
123         } else {
124             printf("Invalid choice.\n");
125         }
126     } while (!unlocked);
127
128     do {
129         printf("\n-------------- LIFEVAULT: %s --------------\n", activeUser.name);
130         printf("1. Update Emergency Profile\n");
131         printf("2. Show Emergency Card\n");
132         printf("3. Add Diary Life Record\n");
133         printf("4. View Diary Timeline\n");
134         printf("5. Analyze Diary Patterns\n");
135         printf("6. Display Vault Report\n");
136         printf("7. Exit\n");
137         printf("Enter your choice: ");
138
139         if (scanf("%d", &choice) != 1) {
140             printf("Invalid input.\n");
141             clearInputBuffer();
142             continue;
143         }
144         clearInputBuffer();
145
146         switch (choice) {
147             case 1:
148                 createEmergencyProfile(&profile);
149                 break;
150             case 2:
151                 showEmergencyProfile(&profile);
152                 break;
153             case 3:
154                 addLifeRecord(records, &recordCount);
155                 break;
156             case 4:
157                 viewLifeTimeline(records, recordCount);
158                 break;
159             case 5:
160                 analyzeDiaryPatterns(records, recordCount);
161                 break;
162             case 6:
163                 displayVaultReport(records, recordCount);
164                 break;
165             case 7:
166                 printf("LifeVault locked. Records saved to %s.\n", FILE_NAME);
167                 break;
168             default:
169                 printf("Invalid choice.\n");
170         }
171     } while (choice != 7);
172
173     return 0;
174 }
175
176 void clearInputBuffer(void) {
177     int ch;
178     while ((ch = getchar()) != '\n' && ch != EOF) {
179     }
180 }
181
```

Line	Code

```
182 void readLine(char text[], int size) {
183     fgets(text, size, stdin);
184     text[strcspn(text, "\n")] = '\0';
185 }
186 
187 void initializeEmergencyProfile(EmergencyProfile *profile) {
188     strcpy(profile->fullName, "Not added");
189     strcpy(profile->bloodGroup, "Not added");
190     strcpy(profile->allergies, "Not added");
191     strcpy(profile->medicalNotes, "Not added");
192     strcpy(profile->contactName, "Not added");
193     strcpy(profile->contactPhone, "Not added");
194     strcpy(profile->routine, "Not added");
195 }
196 
197 int joinVault(VaultUser *user) {
198     VaultUser existingUser;
199 
200     printf("\nName: ");
201     readLine(user->name, NAME_SIZE);
202     printf("Gmail: ");
203     readLine(user->gmail, GMAIL_SIZE);
204     printf("Secret type (passcode/password): ");
205     readLine(user->secretType, sizeof(user->secretType));
206     printf("Create %s: ", user->secretType);
207     readLine(user->secret, SECRET_SIZE);
208 
209     if (strlen(user->name) == 0 || strlen(user->gmail) == 0 || strlen(user->secret) == 0) {
210         printf("Name, Gmail, and secret are required.\n");
211         return 0;
212     }
213 
214     if (findUserByGmail(user->gmail, &existingUser)) {
215         printf("This Gmail already has a vault. Please login instead.\n");
216         return 0;
217     }
218 
219     FILE *file = fopen(PASSCODE_FILE, "a");
220     if (file == NULL) {
221         printf("Could not open %s for saving.\n", PASSCODE_FILE);
222         return 0;
223     }
224 
225     fprintf(file, "%s|%s|%s|%s\n", user->name, user->gmail, user->secretType, user->secret);
226     fclose(file);
227     saveRememberedUser(user->gmail);
228     printf("Vault created. Your login is saved in %s.\n", PASSCODE_FILE);
229     return 1;
230 }
231 
232 int loginRememberedUser(VaultUser *user) {
233     char enteredSecret[SECRET_SIZE];
234 
235     if (!loadRememberedUser(user)) {
236         printf("No remembered user found. Please join or login with Gmail.\n");
237         return 0;
238     }
239 
240     printf("Remembered user: %s (%s)\n", user->name, user->gmail);
241     printf("Enter %s: ", user->secretType);
242     readLine(enteredSecret, SECRET_SIZE);
243 
244     if (strcmp(enteredSecret, user->secret) == 0) {
245         printf("Vault unlocked.\n");
246         return 1;
247     }
248 
249     printf("Incorrect %s.\n", user->secretType);
250     return 0;
251 }
252 
253 int loginWithGmail(VaultUser *user) {
254     char gmail[GMAIL_SIZE];
255     char enteredSecret[SECRET_SIZE];
256 
257     printf("\nGmail: ");
258     readLine(gmail, GMAIL_SIZE);
259 
260     if (!findUserByGmail(gmail, user)) {
261         printf("No vault found for this Gmail.\n");
262         return 0;
263     }
264 
265     printf("Enter %s: ", user->secretType);
266     readLine(enteredSecret, SECRET_SIZE);
267 
268     if (strcmp(enteredSecret, user->secret) == 0) {
269         saveRememberedUser(user->gmail);
270         printf("Vault unlocked.\n");
271         return 1;
272     }
273 
274     printf("Incorrect %s.\n", user->secretType);
275     return 0;
```

Line	Code

```
}

int findUserByGmail(const char gmail[], VaultUser *user) {
    FILE *file = fopen(PASSCODE_FILE, "r");
    char line[260];
    char *name;
    char *storedGmail;
    char *secretType;
    char *secret;

    if (file == NULL) {
        return 0;
    }

    while (fgets(line, sizeof(line), file) != NULL) {
        line[strcspn(line, "\n")] = '\0';
        name = strtok(line, "|");
        storedGmail = strtok(NULL, "|");
        secretType = strtok(NULL, "|");
        secret = strtok(NULL, "|");

        if (name == NULL || storedGmail == NULL || secretType == NULL || secret == NULL) {
            continue;
        }

        if (strcmp(storedGmail, gmail) == 0) {
            strcpy(user->name, name);
            strcpy(user->gmail, storedGmail);
            strcpy(user->secretType, secretType);
            strcpy(user->secret, secret);
            fclose(file);
            return 1;
        }
    }

    fclose(file);
    return 0;
}

void saveRememberedUser(const char gmail[]) {
    FILE *file = fopen(REMEMBERED_FILE, "w");

    if (file == NULL) {
        return;
    }

    fprintf(file, "%s\n", gmail);
    fclose(file);
}

int loadRememberedUser(VaultUser *user) {
    FILE *file = fopen(REMEMBERED_FILE, "r");
    char gmail[GMAIL_SIZE];

    if (file == NULL) {
        return 0;
    }

    if (fgets(gmail, sizeof(gmail), file) == NULL) {
        fclose(file);
        return 0;
    }

    fclose(file);
    gmail[strcspn(gmail, "\n")] = '\0';
    return findUserByGmail(gmail, user);
}

void createEmergencyProfile(EmergencyProfile *profile) {
    printf("\nFull name: ");
    readLine(profile->fullName, NAME_SIZE);
    printf("Blood group: ");
    readLine(profile->bloodGroup, sizeof(profile->bloodGroup));
    printf("Allergies: ");
    readLine(profile->allergies, TEXT_SIZE);
    printf("Medical notes: ");
    readLine(profile->medicalNotes, TEXT_SIZE);
    printf("Emergency contact name: ");
    readLine(profile->contactName, NAME_SIZE);
    printf("Emergency contact phone: ");
    readLine(profile->contactPhone, sizeof(profile->contactPhone));
    printf("Daily routine: ");
    readLine(profile->routine, TEXT_SIZE);
    printf("Emergency profile saved.\n");
}

void showEmergencyProfile(const EmergencyProfile *profile) {
    printf("\n---------- Emergency Card ----------\n");
    printf("Name: %s\n", profile->fullName);
    printf("Blood Group: %s\n", profile->bloodGroup);
    printf("Allergies: %s\n", profile->allergies);
    printf("Medical Notes: %s\n", profile->medicalNotes);
    printf("Emergency Contact: %s\n", profile->contactName);
    printf("Contact Phone: %s\n", profile->contactPhone);
```

Line	Code

```
    printf("Daily Routine: %s\n", profile->routine);
}

void addLifeRecord(LifeRecord records[], int *recordCount) {
    LifeRecord *record;
    char defaultDate[DATE_SIZE];

    if (*recordCount >= MAX_RECORDS) {
        printf("Life record storage is full.\n");
        return;
    }

    record = &records[*recordCount];
    memset(record, 0, sizeof(LifeRecord));
    todayToDmy(defaultDate);

    printf("\nDiary entry title: ");
    readLine(record->title, TITLE_SIZE);
    printf("Entry date (DD-MM-YYYY, blank for today %s): ", defaultDate);
    readLine(record->entryDate, DATE_SIZE);
    if (strlen(record->entryDate) == 0) {
        strcpy(record->entryDate, defaultDate);
    }

    chooseRecordType(record);

    printf("Mood or feeling: ");
    readLine(record->mood, sizeof(record->mood));
    printf("Tags or keywords: ");
    readLine(record->tags, TEXT_SIZE);

    if (strstr(record->type, "Future") != NULL) {
        printf("Unlock date for this future entry (DD-MM-YYYY): ");
        readLine(record->unlockDate, DATE_SIZE);
    } else {
        strcpy(record->unlockDate, "");
    }

    setTypeQuestions(record);

    printf("%s: ", record->questionOne);
    readLine(record->answerOne, TEXT_SIZE);
    printf("%s: ", record->questionTwo);
    readLine(record->answerTwo, TEXT_SIZE);
    printf("%s: ", record->questionThree);
    readLine(record->answerThree, TEXT_SIZE);
    printf("Extra diary note: ");
    readLine(record->diaryBody, TEXT_SIZE);

    (*recordCount)++;
    saveRecordToFile(record);
    printf("Diary life record saved.\n");
}

void chooseRecordType(LifeRecord *record) {
    int typeChoice = 0;

    printf("\nChoose diary type:\n");
    printf("1. Life Event\n");
    printf("2. Future Message\n");
    printf("3. Dream Signal\n");
    printf("4. Legacy Instruction\n");
    printf("5. Memory or Goal\n");
    printf("Enter type number: ");

    if (scanf("%d", &typeChoice) != 1) {
        typeChoice = 1;
    }
    clearInputBuffer();

    switch (typeChoice) {
        case 2:
            strcpy(record->type, "Future Message");
            break;
        case 3:
            strcpy(record->type, "Dream Signal");
            break;
        case 4:
            strcpy(record->type, "Legacy Instruction");
            break;
        case 5:
            strcpy(record->type, "Memory or Goal");
            break;
        default:
            strcpy(record->type, "Life Event");
            break;
    }
}

void setTypeQuestions(LifeRecord *record) {
    if (strstr(record->type, "Future") != NULL) {
        strcpy(record->questionOne, "What should future you read");
        strcpy(record->questionTwo, "Why should this open later");
        strcpy(record->questionThree, "What reminder should it carry");
```

```
    } else if (strstr(record->type, "Dream") != NULL) {
        strcpy(record->questionOne, "What did you see in the dream");
        strcpy(record->questionTwo, "How did it feel after waking up");
        strcpy(record->questionThree, "Which symbols or words repeated");
    } else if (strstr(record->type, "Legacy") != NULL) {
        strcpy(record->questionOne, "Who is this meant for");
        strcpy(record->questionTwo, "What message or instruction should remain");
        strcpy(record->questionThree, "Why is this important");
    } else if (strstr(record->type, "Memory") != NULL || strstr(record->type, "Goal") != NULL) {
        strcpy(record->questionOne, "What memory or goal do you want to keep");
        strcpy(record->questionTwo, "What made it meaningful");
        strcpy(record->questionThree, "What next step or promise belongs with it");
    } else {
        strcpy(record->questionOne, "What happened");
        strcpy(record->questionTwo, "Why did it matter");
        strcpy(record->questionThree, "What did you learn or decide");
    }
}

void viewLifeTimeline(LifeRecord records[], int recordCount) {
    int i;

    if (recordCount == 0) {
        printf("No diary entries saved.\n");
        return;
    }

    printf("\n---------- Diary Timeline ----------\n");
    for (i = 0; i < recordCount; i++) {
        printf("\n%d. %s [%s]\n", i + 1, records[i].title, records[i].type);
        printf("Entry Date: %s | Mood: %s\n", records[i].entryDate, records[i].mood);
        printf("Tags: %s\n", records[i].tags);

        if (isRecordLocked(&records[i])) {
            printf("Diary entry: Locked until %s\n", records[i].unlockDate);
        } else {
            printf("%s: %s\n", records[i].questionOne, records[i].answerOne);
            printf("%s: %s\n", records[i].questionTwo, records[i].answerTwo);
            printf("%s: %s\n", records[i].questionThree, records[i].answerThree);
            printf("Extra Diary Note: %s\n", records[i].diaryBody);
        }
    }
}

void analyzeDiaryPatterns(const LifeRecord records[], int recordCount) {
    int i;
    int lockedCount = 0;
    int revisitCount = 0;
    int taggedCount = 0;

    if (recordCount == 0) {
        printf("No diary entries to analyze.\n");
        return;
    }

    printf("\n---------- Diary Pattern Analyzer ----------\n");
    for (i = 0; i < recordCount; i++) {
        if (isRecordLocked(&records[i])) {
            lockedCount++;
        }

        if (strlen(records[i].tags) > 0) {
            taggedCount++;
        }

        if (!isRecordLocked(&records[i]) && isReflectiveRecord(&records[i])) {
            revisitCount++;
            printf("Revisit suggestion: \"%s\" has a strong feeling or warning keyword.\n", records[i].title);
        }
    }

    printf("\nTotal diary entries: %d\n", recordCount);
    printf("Entries with tags: %d\n", taggedCount);
    printf("Locked future entries: %d\n", lockedCount);
    printf("Entries to revisit: %d\n", revisitCount);

    if (revisitCount > 0) {
        printf("Main signal: some diary entries may need reflection later.\n");
    } else {
        printf("Main signal: no urgent revisit pattern found yet.\n");
    }
}

void displayVaultReport(const LifeRecord records[], int recordCount) {
    int i;
    int futureCount = 0;
    int dreamCount = 0;
    int legacyCount = 0;
    int memoryCount = 0;
    int eventCount = 0;

    for (i = 0; i < recordCount; i++) {
        if (strstr(records[i].type, "Future") != NULL) {
            futureCount++;
```

Line	Code

```
        } else if (strstr(records[i].type, "Dream") != NULL) {
            dreamCount++;
        } else if (strstr(records[i].type, "Legacy") != NULL) {
            legacyCount++;
        } else if (strstr(records[i].type, "Memory") != NULL || strstr(records[i].type, "Goal") != NULL) {
            memoryCount++;
        } else {
            eventCount++;
        }
    }

    printf("\n---------- Vault Report ----------\n");
    printf("Total diary life records: %d\n", recordCount);
    printf("Life events: %d\n", eventCount);
    printf("Future messages: %d\n", futureCount);
    printf("Dream signals: %d\n", dreamCount);
    printf("Legacy instructions: %d\n", legacyCount);
    printf("Memories or goals: %d\n", memoryCount);
}

void saveRecordToFile(const LifeRecord *record) {
    FILE *file = fopen(FILE_NAME, "a");

    if (file == NULL) {
        printf("Warning: Could not save record to file.\n");
        return;
    }

    fprintf(file, "[Diary Life Record]\n");
    fprintf(file, "Title: %s\n", record->title);
    fprintf(file, "Type: %s\n", record->type);
    fprintf(file, "Entry Date: %s\n", record->entryDate);
    fprintf(file, "Mood: %s\n", record->mood);
    fprintf(file, "Tags: %s\n", record->tags);
    if (strlen(record->unlockDate) > 0) {
        fprintf(file, "Unlock Date: %s\n", record->unlockDate);
    }
    fprintf(file, "%s: %s\n", record->questionOne, record->answerOne);
    fprintf(file, "%s: %s\n", record->questionTwo, record->answerTwo);
    fprintf(file, "%s: %s\n", record->questionThree, record->answerThree);
    fprintf(file, "Extra Diary Note: %s\n\n", record->diaryBody);
    fclose(file);
}

int isReflectiveRecord(const LifeRecord *record) {
    if (containsIgnoreCase(record->mood, "anxious") ||
        containsIgnoreCase(record->mood, "stressed") ||
        containsIgnoreCase(record->mood, "angry") ||
        containsIgnoreCase(record->mood, "low") ||
        containsIgnoreCase(record->mood, "confused")) {
        return 1;
    }

    if (containsIgnoreCase(record->tags, "warning") ||
        containsIgnoreCase(record->tags, "health") ||
        containsIgnoreCase(record->answerOne, "regret") ||
        containsIgnoreCase(record->answerTwo, "mistake") ||
        containsIgnoreCase(record->answerThree, "urgent") ||
        containsIgnoreCase(record->diaryBody, "hurt")) {
        return 1;
    }

    return 0;
}

int containsIgnoreCase(const char text[], const char keyword[]) {
    int i;
    int j;
    int textLength = (int)strlen(text);
    int keywordLength = (int)strlen(keyword);

    if (keywordLength == 0 || keywordLength > textLength) {
        return 0;
    }

    for (i = 0; i <= textLength - keywordLength; i++) {
        for (j = 0; j < keywordLength; j++) {
            if (tolower((unsigned char)text[i + j]) != tolower((unsigned char)keyword[j])) {
                break;
            }
        }

        if (j == keywordLength) {
            return 1;
        }
    }

    return 0;
}

int isRecordLocked(const LifeRecord *record) {
    if (strlen(record->unlockDate) == 0) {
        return 0;
    }
```

Line	Code

```
652
653     return todayToNumber() < dateToNumber(record->unlockDate);
654 }
655
656 int dateToNumber(const char date[]) {
657     int day = 0;
658     int month = 0;
659     int year = 0;
660
661     if (sscanf(date, "%d-%d-%d", &day, &month, &year) != 3) {
662         return 0;
663     }
664
665     return year * 10000 + month * 100 + day;
666 }
667
668 int todayToNumber(void) {
669     time_t currentTime = time(NULL);
670     struct tm *today = localtime(&currentTime);
671
672     return (today->tm_year + 1900) * 10000 + (today->tm_mon + 1) * 100 + today->tm_mday;
673 }
674
675 void todayToDmy(char date[]) {
676     time_t currentTime = time(NULL);
677     struct tm *today = localtime(&currentTime);
678
679     sprintf(date, "%02d-%02d-%04d", today->tm_mday, today->tm_mon + 1, today->tm_year + 1900);
680 }
```